title: WebSocket Gateway 对接指南 weight: 1 description: 面向第三方开发者，从快速上手到高级特性，完整介绍 HotPlex WebSocket Gateway 对接方法

WebSocket Gateway 对接指南

快速上手：30 秒跑通

```
// 1. 连接
const ws = new WebSocket('ws://localhost:8080/ws');

ws.onopen = () => {
  // 2. 握手（必须作为第一帧）
  ws.send(JSON.stringify({
    version: 'aep/v1',
    id: crypto.randomUUID(),
    session_id: '',                          // 空字符串，自动派生
    seq: 0,
    timestamp: Date.now(),
    event: {
      type: 'init',
      data: {
        version: 'aep/v1',
        worker_type: 'claude_code',
        auth: { token: 'your-api-key' }
      }
    }
  }) + '\n');
};

ws.onmessage = (e) => {
  const msg = JSON.parse(e.data);
  const type = msg.event.type;

  if (type === 'init_ack') {
    // 3. 握手成功，可以发消息了
    console.log('Session:', msg.session_id, 'State:', msg.event.data.state);
    sendInput('你好，请介绍一下你自己');
  } else if (type === 'message.delta') {
    // 4. 流式输出（逐字到达）
    process.stdout.write(msg.event.data.content);
  } else if (type === 'message') {
```

```
    // 5. 完整回复（Turn 聚合）
    console.log('\n[完整回复]', msg.event.data.content);
  } else if (type === 'done') {
    // 6. Turn 结束，可以发下一条
    console.log('[Turn 完成]');
    // sendInput('下一个问题');
  }
};

function sendInput(text) {
  ws.send(JSON.stringify({
    version: 'aep/v1',
    id: crypto.randomUUID(),
    session_id: '', seq: 0, timestamp: Date.now(),
    event: { type: 'input', data: { content: text } }
  }) + '\n');
}
```

运行上面的代码，你就能看到 AI 的流式回复。以下是完整的工作原理。

核心概念

通信模型

```
你的客户端  ◀──── WebSocket（全双工，NDJSON）────▶  HotPlex Gateway  ◀── stdio ──▶  AI Worker
```

- **传输**：WebSocket，每条消息是一个 JSON 对象（NDJSON 格式）
- **协议**：AEP v1（Agent Event Protocol），统一信封格式
- **模式**：全双工，客户端和服务端可以同时发送消息

消息信封

所有消息都遵循相同的结构：

```
{
  "version": "aep/v1",
  "id": "evt_550e8400-...",
  "session_id": "a1b2c3d4-...",
  "seq": 0,
  "timestamp": 1710000000000,
  "event": {
    "type": "input",
```

```
    "data": { "content": "你好" }
  }
}
```

字段	说明
`version`	固定 `"aep/v1"`
`id`	消息唯一 ID，任意 UUID 即可
`session_id`	会话 ID（见下方 Session 章节）
`seq`	序列号，客户端发 `0`，Gateway 自动分配
`timestamp`	Unix 毫秒时间戳
`event.type`	消息类型
`event.data`	消息载荷

一次完整对话的流程

```
客户端                              Gateway                          AI Worker
  |                                   |                                 |
  |  ① WS 连接                        |                                 |
  |──────────────────────────────────>|                                 |
  |  ② init {auth, worker_type}       |                                 |
  |──────────────────────────────────>|  创建 Session + 启动 Worker     |
  |  ③ init_ack {session_id, state}   |                                 |
  |<──────────────────────────────────|                                 |
  |                                   |                                 |
  |  ④ input {content: "你好"}        |                                 |
  |──────────────────────────────────>|  转发给 Worker ────────────────>|
  |                                   |                                 |
  |  ⑤ message.start                  |                                 |
  |<─ ⑥ message.delta (逐字)          |  Worker 流式输出 <──────────────|
  |<─ ⑦ message.delta                 |                                 |
  |<─ ⑧ message.end                   |                                 |
  |<─ ⑨ message {完整文本}            |                                 |
  |<─ ⑩ done {success: true}          |                                 |
  |<──────────────────────────────────|                                 |
  |                                   |                                 |
  |  可以继续发送下一条 input ...      |                                 |
```

连接与认证

连接端点

```
ws://<host>:<port>/ws
```

两种认证方式

API Key — 简单快速，适合单用户/内部服务

在 HTTP Header 中携带：

```
curl -i --no-buffer \
  -H "X-API-Key: your-api-key" \
  -H "Upgrade: websocket" \
  -H "Connection: Upgrade" \
  http://localhost:8080/ws
```

浏览器无法自定义 WS Header，可在 init 信封中携带：

```
{ "event": { "type": "init", "data": {
  "version": "aep/v1",
  "worker_type": "claude_code",
  "auth": { "token": "your-api-key" }
}}}
```

JWT — 多用户隔离，适合 SaaS 产品

签发 ES256 签名的 JWT，在 init 信封中携带：

```
{ "event": { "type": "init", "data": {
  "version": "aep/v1",
  "worker_type": "claude_code",
  "auth": { "token": "eyJhbGciOiJFUzI1NiIs..." }
}}}
```

JWT 关键 Claims：

字段	必填	说明
`sub`	是	用户唯一标识（如 `"user-123"`），用于会话隔离
`bot_id`	否	Bot 身份标识，用于多 Bot 隔离
`exp`	是	过期时间
`iss`	是	固定 `"hotplex"`

如何选择

	API KEY	JWT
用户身份	全部为 `api_user`（共享）	`sub` claim（每人独立）
会话隔离	无用户级隔离	按用户隔离
适用场景	单用户/内部测试	多用户 SaaS

> **多用户场景必须用 JWT**。API Key 认证下所有用户共享 `api_user` 身份，无法区分。

Session 管理

Session 是什么

Session 代表一个独立的对话上下文。每个 Session 绑定一个 Worker 进程，拥有独立的状态和对话历史。

Session ID 如何确定

Gateway 使用 **UUIDv5 确定性派生**，从四个维度生成唯一 ID：

```
Session ID = UUIDv5(userID | workerType | clientSessionID | workDir)
```

clientSessionID 是什么：你在 init 信封 `session_id` 字段传入的值。它**不是** Session ID 本身，只是派生函数的一个输入。

三个关键规则

规则 1：不传 clientSessionID → 自动获得固定会话

```
{ "session_id": "", "event": { "type": "init", "data": { "version": "aep/v1", "worker_type": "claude_code"
} } }
```

相同的 (userID, workerType, "", workDir) → 永远同一个 Session ID。断线重连自动恢复。

规则 2：传 clientSessionID → 多会话并行

```
// 每个浏览器 tab 生成独立 ID
const tabId = `sess_${crypto.randomUUID()}`;
// init 时传入 → 每个 tab 独立会话
```

规则 3：重连时必须重传原始 clientSessionID

```
首次:  init { session_id: "sess_tab-abc" }
       → 派生得到 "a1b2c3d4-..." → init_ack.session_id = "a1b2c3d4-..."

重连:  init { session_id: "sess_tab-abc" }     ← 重传同一个
       → 再次派生得到 "a1b2c3d4-..." (确定性！) → 自动恢复
```

> 如果错误地传 Gateway 返回的 `"a1b2c3d4-..."`，在 Session 被 GC 删除后，派生结果会变，导致创建全新会话、对话历史丢失。

两个 ID 各自的用途：

ID	用在哪
clientSessionID（你自己生成的）	init 握手时传入，重连时重传
Gateway Session ID（init_ack 返回的）	REST API 调用（查询历史、删除会话等）

Session 状态

```
CREATED ──→ RUNNING ──→ IDLE ──→ TERMINATED ──→ DELETED
   |                    ▲  ▲          ▲
   └── Worker 启动 ────┘  └── resume ┘
```

状态	含义	能发消息吗
`running`	Worker 正在执行	能
`idle`	Worker 空闲，等新输入	能（自动切换到 running）
`terminated`	Worker 已终止	需重连恢复
`deleted`	终态，已删除	不能

会话恢复决策

init 握手时，Gateway 根据 Session 状态自动决策：

状态	GATEWAY 行为
Session 不存在	创建新会话 + 启动 Worker
Worker 仍存活	**Fast Reconnect** — 直接复用，零延迟
idle / terminated	Resume — 恢复对话历史
Resume 失败	降级创建新会话

消息收发

发送用户输入

```
{
  "version": "aep/v1",
  "id": "evt_...",
  "session_id": "",
  "seq": 0,
  "timestamp": 1710000001000,
  "event": {
    "type": "input",
    "data": { "content": "请帮我分析这段代码" }
  }
}
```

> `session_id` 和 `seq` 由 Gateway 覆盖，客户端填空字符串或 `0` 即可。

限制：Session 必须处于 Active 状态（running / idle），否则返回 `SESSION_BUSY` 错误。

接收流式响应

一个完整 Turn 的事件序列：

```
message.start      ← 开始输出
message.delta × N  ← 逐字流式（高频，可能被丢弃）
message.end        ← 结束输出
message            ← 完整文本聚合
done               ← Turn 终止符
```

message.delta — 增量文本

```
{ "event": { "type": "message.delta", "data": { "message_id": "msg_1", "content": "根据" } } }
{ "event": { "type": "message.delta", "data": { "message_id": "msg_1", "content": "代码分析" } } }
```

拼接所有 delta 即可获得流式效果。delta 可能因背压被丢弃，详见背压与丢弃。

message — 完整文本

```
{ "event": { "type": "message", "data": {
  "id": "msg_1",
  "role": "assistant",
  "content": "根据代码分析，主要瓶颈在..."
} } }
```

Turn 结束时发送，包含完整的回复文本。**如果 delta 被丢弃，以 message 为准。**

done — Turn 结束

```
{ "event": { "type": "done", "data": {
  "success": true,
  "stats": { "total_tokens": 1500, "duration_ms": 3200 }
} } }
```

收到 `done` 后可以发送下一条 input。如果期间有 delta 被丢弃，`dropped` 字段为 `true`。

辅助事件

Worker 执行过程中还可能产生：

事件	说明	何时出现
`tool_call`	调用工具（读文件、执行命令等）	Worker 使用工具时
`tool_result`	工具执行结果	工具完成后
`reasoning`	推理过程	Worker 思考时（取决于配置）
`step`	执行步骤	Worker 分步执行时

用户交互

Worker 执行过程中可能需要用户参与。三种交互类型都遵循相同的模式：**Gateway 发送请求 → 客户端通过 input 响应**。

权限确认 — Worker 请求执行工具

```
// Gateway → 客户端
{ "event": { "type": "permission_request", "data": {
  "id": "perm_1",
  "tool_name": "Bash",
  "input_raw": "{\"command\":\"rm -rf /tmp/*\"}"
} } }

// 客户端 → Gateway（允许）
{ "event": { "type": "input", "data": {
  "content": "yes",
  "metadata": { "permission_response": { "id": "perm_1", "allowed": true } }
} } }

// 客户端 → Gateway（拒绝）
{ "event": { "type": "input", "data": {
  "content": "",
  "metadata": { "permission_response": { "id": "perm_1", "allowed": false, "reason": "不允许" } }
} } }
```

问答请求 — Worker 需要用户选择

```
// Gateway → 客户端
{ "event": { "type": "question_request", "data": {
  "id": "q_1",
  "questions": [{ "question": "选择环境", "header": "环境", "options": [
    { "label": "staging", "description": "预发布" },
    { "label": "production", "description": "生产" }
  ], "multi_select": false }]
} } }

// 客户端 → Gateway
{ "event": { "type": "input", "data": {
  "content": "staging",
  "metadata": { "question_response": { "id": "q_1", "answers": { "选择环境": "staging" } } }
} } }
```

MCP 输入请求 — MCP Server 需要用户信息

```
// Gateway → 客户端
{ "event": { "type": "elicitation_request", "data": {
  "id": "el_1",
  "mcp_server_name": "github",
  "message": "请输入 GitHub Token"
} } }

// 客户端 → Gateway
{ "event": { "type": "input", "data": {
  "content": "",
  "metadata": { "elicitation_response": { "id": "el_1", "action": "accept", "content": { "token": "ghp_xx
x" } } }
} } }
```

超时：所有交互默认 5 分钟，超时后自动拒绝（auto-deny）。

心跳保活

项目	值
Server Ping 间隔	54 秒（WebSocket Ping 帧）

项目	值
Pong 超时	60 秒未回复则断开
客户端主动 Ping	可发 AEP `ping` 事件，Gateway 回复 `pong`（含当前 session state）

```
// 客户端 → Gateway
{ "version": "aep/v1", "id": "evt_...", "session_id": "", "seq": 0,
  "timestamp": 1710000005000,
  "event": { "type": "ping", "data": {} } }

// Gateway → 客户端（seq=0，不消耗序列号）
{ "event": { "type": "pong", "data": { "state": "running" } } }
```

断线重连

重连步骤

1. WebSocket 断开后，等待指数退避时间（1s, 2s, 4s, 8s...最大 30s）
2. 重新建立 WebSocket 连接
3. 发送 init，**携带与首次完全相同的参数**（clientSessionID、auth、workDir）
4. Gateway 派生出相同的 Session ID → 自动恢复

完整重连示例

```
class HotPlexClient {
  constructor(url, auth, workDir) {
    this.url = url;
    this.auth = auth;
    this.workDir = workDir;

    // clientSessionID: 持久化保存，重连时重传以恢复同一会话
    this.clientSessionId = localStorage.getItem('hp_sid')
      || `sess_${crypto.randomUUID()}`;
    localStorage.setItem('hp_sid', this.clientSessionId);

    // gatewaySessionId: init_ack 返回，用于 REST API
    this.gatewaySessionId = null;

    this.reconnectAttempts = 0;
```

```
    this.maxReconnectAttempts = 10;
  }

  connect() {
    this.ws = new WebSocket(`${this.url}/ws`);
    this.ws.onopen = () => this.sendInit();
    this.ws.onmessage = (e) => {
      const env = JSON.parse(e.data);
      if (env.event.type === 'init_ack') {
        if (env.event.data.error) { console.error('Init failed:', env.event.data.code); return; }
        this.gatewaySessionId = env.session_id;  // 保存用于 REST API
        this.reconnectAttempts = 0;
      }
      this.onMessage?.(env);
    };
    this.ws.onclose = () => this.scheduleReconnect();
  }

  sendInit() {
    this.ws.send(JSON.stringify({
      version: 'aep/v1',
      id: `evt_${crypto.randomUUID()}`,
      session_id: this.clientSessionId,  // ← 重传保存的 clientSessionID
      seq: 0, timestamp: Date.now(),
      event: { type: 'init', data: {
        version: 'aep/v1', worker_type: 'claude_code',
        auth: { token: this.auth },
        config: { work_dir: this.workDir }
      }}
    }) + '\n');
  }

  sendInput(content) {
    this.ws.send(JSON.stringify({
      version: 'aep/v1',
      id: `evt_${crypto.randomUUID()}`,
      session_id: '', seq: 0, timestamp: Date.now(),
      event: { type: 'input', data: { content } }
    }) + '\n');
  }

  scheduleReconnect() {
    if (this.reconnectAttempts >= this.maxReconnectAttempts) return;
    const delay = Math.min(1000 * 2 ** this.reconnectAttempts, 30000);
    this.reconnectAttempts++;
    setTimeout(() => this.connect(), delay);
  }
}
```

会话隔离

四维度隔离

Session ID 由四个维度派生，任何维度不同都会产生不同的 Session：

维度	说明	隔离效果
userID	JWT `sub`（或 API Key 的 `api_user`）	不同用户 → 不同会话
workerType	`claude_code` 等	不同引擎 → 不同会话
clientSessionID	客户端生成的 ID	不同 tab → 不同会话
workDir	工作目录	不同项目 → 不同会话

多用户隔离

使用 API Key 认证时所有用户都是 `api_user` → **无法隔离**。使用 JWT 认证，`sub` 参与派生 → **天然隔离**：

```
Alice JWT {sub:"alice"} → "alice|claude_code|tab-1|/project" → Session A
Bob   JWT {sub:"bob"}   → "bob|claude_code|tab-1|/project"   → Session B
```

`ListSessions` API 按 userID 过滤，每个用户只看到自己的会话。

多 Tab 隔离

每个 tab 生成独立的 clientSessionID：

```
// 每个 tab 独立 ID
const tabId = `sess_${crypto.randomUUID()}`;
```

如果两个 tab 用相同的 clientSessionID，后加入的 tab 接管会话，先前的不再收到消息。

控制命令

通过 `control` 事件管理会话：

```
{ "event": { "type": "control", "data": { "action": "terminate" } } }  // 终止
{ "event": { "type": "control", "data": { "action": "reset" } } }      // 重置上下文
{ "event": { "type": "control", "data": { "action": "delete" } } }      // 删除
{ "event": { "type": "control", "data": { "action": "cd", "details": { "path": "/new/dir" } } }  // 切换目录
```

也可在 `input.content` 中用快捷命令：

命令	效果
`/reset`	重置会话上下文
`/gc`	回收空闲会话

背压与丢弃

当客户端消费速度跟不上 Worker 输出时：

事件类型	策略
`message.delta`	**可丢弃** — 通道满时静默丢弃
`raw`	**可丢弃**
`state`，`done`，`error`，`message`	**保障送达** — 阻塞等待

客户端处理：收到 `done` 时检查 `dropped` 字段，如果为 `true`，用 `message` 中的完整文本替代拼接的 delta。

会话历史查询

Gateway 提供两个 REST API 查询历史，需要认证且校验 session 归属（非 owner 返回 403）。

Turn 级别 — 聊天记录

适合展示对话列表（一句提问一句回答）。

```
curl -H "X-API-Key: your-key" \
  "http://localhost:8080/api/sessions/{session_id}/history?limit=50"
```

参数：`limit`（1-200，默认 50）、`before_seq`（游标翻页）

响应：

```
{
  "records": [
    {
      "seq": 15,
      "role": "user",
      "content": "解释这个函数",
      "created_at": 1710000000
    },
    {
      "seq": 28,
      "role": "assistant",
      "content": "这个函数的作用是...",
      "model": "claude-sonnet-4-6",
      "success": true,
      "tools": {"Read": 1, "Bash": 1},
      "tool_call_count": 2,
      "tokens_in": 1200,
      "tokens_out": 350,
      "duration_ms": 3200,
      "cost_usd": 0.012,
      "created_at": 1710000003
    }
  ],
  "has_more": false
}
```

翻页：`has_more` 为 `true` 时，用最后一条的 `seq` 请求下一页：

```
curl "http://localhost:8080/api/sessions/{id}/history?limit=20&before_seq=15"
```

Event 级别 — 原始事件流

适合调试、审计、回放完整会话状态。

```
curl -H "X-API-Key: your-key" \
  "http://localhost:8080/api/sessions/{session_id}/events?limit=200&direction=latest"
```

参数：`limit`（1-1000）、`cursor`（seq 值）、`direction`（`latest` / `before` / `after`）

响应：

```
{
  "events": [
    { "seq": 1, "type": "state", "data": {"state":"running"}, "direction": "outbound" },
    { "seq": 2, "type": "input", "data": {"content":"你好"}, "direction": "inbound" },
    { "seq": 3, "type": "message.delta", "data": {"content":"你好！"}, "direction": "outbound" },
    { "seq": 42, "type": "done", "data": {"success":true}, "direction": "outbound" }
  ],
  "oldest_seq": 1,
  "newest_seq": 42,
  "has_older": false
}
```

三种翻页：

```
direction=latest&cursor=0     # 初始加载：最新 N 条
direction=before&cursor=5     # 向前翻页：seq < 5
direction=after&cursor=42     # 向后追赶：seq > 42
```

如何选择

场景	用哪个
聊天界面展示对话	`/history`
Token 用量统计	`/history`（assistant turn 已聚合）
调试/审计/回放	`/events`
加载更多历史	`/history` + `before_seq`

Init 握手详解

WebSocket 连接建立后，**必须在 30 秒内**发送 `init` 作为第一帧。

init 完整字段

```
{
  "version": "aep/v1",
  "id": "evt_550e8400-...",
  "session_id": "sess_6ba7b810-...",
  "seq": 0,
  "timestamp": 1710000000000,
  "event": {
    "type": "init",
    "data": {
      "version": "aep/v1",                          // 必填，固定 aep/v1
      "worker_type": "claude_code",                  // 必填
      "auth": { "token": "your-api-key-or-jwt" },   // 视场景
      "config": {
        "work_dir": "/home/user/project",            // 可选，工作目录
        "allowed_tools": ["Bash", "Read", "Write"],  // 可选，限制工具
        "system_prompt": "...",                       // 可选，自定义提示词
        "model": "claude-sonnet-4-6"                 // 可选，指定模型
      }
    }
  }
}
```

init_ack 响应

成功时：

```
{
  "session_id": "a1b2c3d4-e5f6-...",
  "state": "running",
  "server_caps": {
    "protocol_version": "aep/v1",
    "supports_resume": true,
    "supports_delta": true,
    "max_frame_size": 32768
  }
}
```

失败时：

```
{ "session_id": "sess_xxx", "error": "version mismatch", "code": "VERSION_MISMATCH" }
```

错误码	原因
`VERSION_MISMATCH`	version 不是 `aep/v1`
`PROTOCOL_VIOLATION`	第一帧不是 init
`UNAUTHORIZED`	认证失败
`RATE_LIMITED`	握手频率过高

连接限制

项目	值
最大消息大小	32 KB
Init 握手超时	30 秒
Pong 检测超时	60 秒
Server Ping 间隔	54 秒
交互确认超时	5 分钟

错误码参考

错误码	说明	建议
`VERSION_MISMATCH`	协议版本不匹配	检查 version 字段
`PROTOCOL_VIOLATION`	协议违规	首帧必须是 init
`INVALID_MESSAGE`	消息格式错误	检查 JSON 结构
`UNAUTHORIZED`	认证失败	检查 API Key / JWT
`SESSION_NOT_FOUND`	Session 不存在	重新 init

错误码	说明	建议
`SESSION_BUSY`	Session 非活跃	等待或重连
`SESSION_EXPIRED`	已过期	创建新会话
`RATE_LIMITED`	频率过高	退避重试
`WORKER_CRASH`	Worker 崩溃	Gateway 自动恢复
`TURN_TIMEOUT`	单轮超时	简化任务
`INTERNAL_ERROR`	服务端错误	查看日志
`RECONNECT_REQUIRED`	服务端要求重连	执行重连

常见问题

多个浏览器 tab 消息串了?

每个 tab 生成独立的 `clientSessionID`（`crypto.randomUUID()`），UUIDv5 会派生出不同的 Session ID。

ListSessions 返回所有用户的会话?

使用 JWT 认证。API Key 认证的身份统一为 `api_user`，无法区分用户。

重连后对话历史丢失?

重连时必须重传与首次完全相同的参数：clientSessionID、auth token、workDir。参数不同会派生出不同的 Session ID。

收到 `dropped: true`?

delta 事件被背压丢弃。用 `message` 事件中的完整文本替代拼接的 delta。

Worker 崩溃了?

Gateway 自动处理：尝试 Resume → 失败则 Fresh Start → 通知客户端。客户端只需正常处理 `error` 事件。

如何查看历史记录?

Turn 级别用 `GET /api/sessions/{id}/history`，Event 级别用 `GET /api/sessions/{id}/events`。详见会话历史查询。